Contents

BeebPerf

BeebPerf is a profiling tool for the BBC Micro and Master, aimed at programmers who want to improve the performance and visual quality of their games/programs written in assembly.

The profiler can help programmers understand…

1. Where their code is spending most of its time. The profiler identifies a program's **hot routines** and **call paths**, for the selected time range.

   Focusing optimization efforts in these areas will likely deliver the largest speedup improvements. And better still, one can re-measure to understand the actual impact of the optimizations made.

2. Which memory addresses are accessed the most, and whether these are zero-page addresses or not, for the selected time range.

   Reviewing the most frequently accessed memory addresses may reveal surprises and opportunities for optimizing how zero-page is used, reducing cycles and freeing up code space.

3. Whether game-loop and rendering code is aligned with v-sync, and whether there are potential screen tearing issues.

   Understanding how a game-loop behaves across iterations can be very useful. The profiler can highlight and report iterations that took longer than 1/50th of a second and can identify cases where the game-loop loses alignment with v-sync (Vertical Sync). It can also highlight iterations where writes to screen memory are not consistently either all before, or all after, the CRTC's (Cathode Ray Tube Controler) memory scan, which displays the screen memory on the CRT display.

4. What the CRT display frames look like and whether these are as expected.

   Examining individual display frames can be useful in identifying rendering issues and potential bugs. One can also review the executed instructions that generated problematic frames.

5. How specific routine durations change over time.

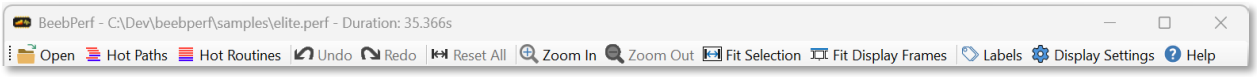
   Understanding how a routine's duration varies over time can be useful in some situations.

Getting started

Click on the **Open** toolbar button to load a **.perf** file created with an augmented version of **BeebEm** (see docs/BeebPerf.docx). Some sample **.perf** files can be found in the **samples** folder.

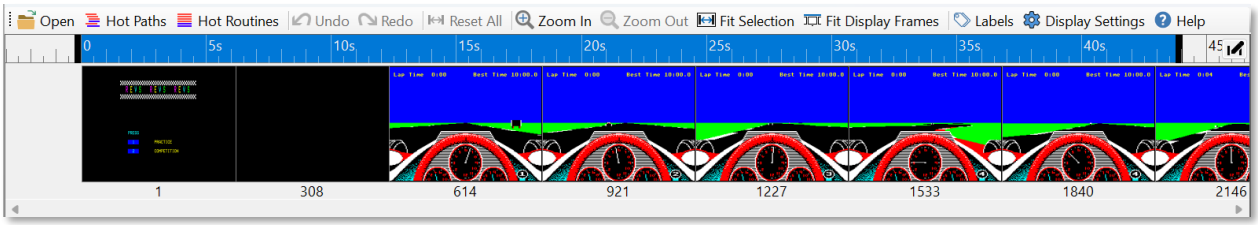
Familiarize yourself with the general layout of the application.

The toolbar...



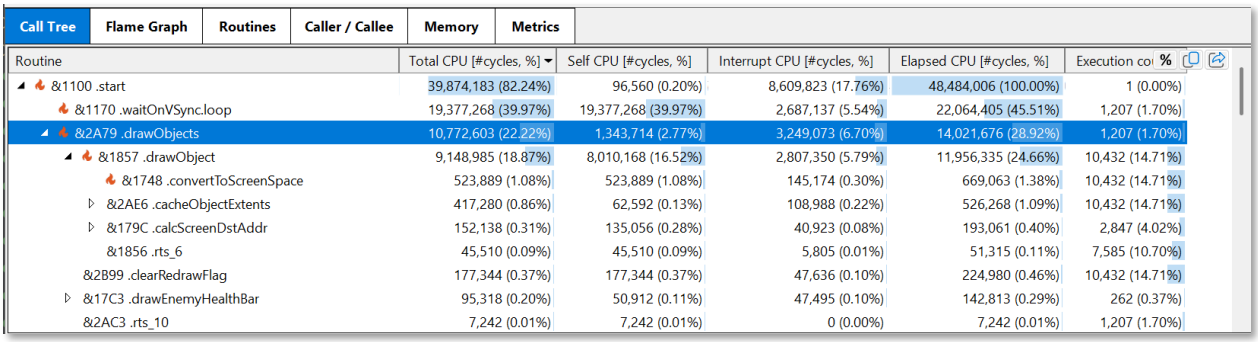
Provides access to all the common actions. We'll explore these in more detail below.

The timeline...



Allows you to select the selected analysis time range, shown as a blue bar. This can be changed by dragging the end handles or clicking on the  button. The display frames are drawn underneath. Toolbar buttons allow you to zoom in and out, rescale the timeline to fit the selected time range, tile the display frames across the window, or reset the selected time range and zoom.

The tabs...



The tabs provide different profiler views, allowing one to look at different aspects of the program. We'll look at each tab in more detail later, but for now, understanding some of the terminology will be useful.

Common routine related terms

Term	Description
Self CPU	The number of cycles spent executing instructions within the routine itself. Excludes cycles spent in other routines or interrupt code.
Total CPU	The number of cycles spent executing instructions within the routine, and any routines it invokes directly or indirectly, from JSR's, tail-calls, and/or fall-throughs. Excludes cycles spent executing interrupt code during the execution of the routine. **Total CPU >= Self CPU**
Interrupt CPU	The number of cycles spent executing interrupt code during the execution of the routine.

Elapsed CPU	The number of cycles spent executing instructions within the routine, or any routines it invokes directly or indirectly, from JSR's, tail-calls, and/or fall-throughs, plus cycles spent executing interrupt code during the execution f the routine. **Elapsed CPU = Total CPU + Interrupt CPU**
Execution count	The number of times the routine was called/invoked.
Tail Call	A **tail call** is a JMP or branch instruction that jumps to another routine. For a JMP, it's equivalent to a **JSR *routine* : RTS**, but uses less cycles and code bytes. For a branch it's an optimized way to perform conditional **JSR *routine*: RTS**.
Fall through	A **fall-though** is where a routine does not return, and execution falls through the end of the routine to the next routine in memory.

The code panel...

Address	Label	Instruction	Tail call	Total CPU [#cycles, %]	Self CPU [#cycles, %]	Branch count [#, %]	Execution count [#, %]
&2B79	.frameCallback_IntermittentRedraw	TXA		49,656 (9.55%)	49,656 (9.55%)		24,828 (100.00%)
&2B7A		CLC		49,656 (9.55%)	49,656 (9.55%)		24,828 (100.00%)
&2B7B		ADC &2C .frameCount		74,484 (14.32%)	74,484 (14.32%)		24,828 (100.00%)
&2B7D		AND #&07		49,656 (9.55%)	49,656 (9.55%)		24,828 (100.00%)
&2B7F		BNE &2BB4 .rts_0	Yes	201,738 (38.79%)	71,382 (13.73%)	21,726 (87.51%)	24,828 (100.00%)
&2B81		CPX &28 .enemyCount		9,306 (1.79%)	9,306 (1.79%)		3,102 (12.49%)
&2B83		BCC &2B8E .setRedrawFlag	Yes	19,284 (3.71%)	6,858 (1.32%)	654 (21.08%)	3,102 (12.49%)
&2B85		BEQ &2B8E .setRedrawFlag		4,896 (0.94%)	4,896 (0.94%)	0 (0.00%)	2,448 (9.86%)
&2B87		LDA &0430 .objectState,X		9,792 (1.88%)	9,792 (1.88%)		2,448 (9.86%)
&2B8A		AND #&10		4,896 (0.94%)	4,896 (0.94%)		2,448 (9.86%)
&2B8C		BNE &2BB4 .rts_0	Yes	7,640 (1.47%)	5,288 (1.02%)	392 (16.01%)	2,448 (9.86%)
			fall-through				
&2B8E	.setRedrawFlag	LDA &0430 .objectState,X		8,224 (1.58%)	8,224 (1.58%)		2,056 (8.28%)
&2B91		ORA #&80		4,112 (0.79%)	4,112 (0.79%)		2,056 (8.28%)
&2B93		STA &0430 .objectState,X		10,280 (1.98%)	10,280 (1.98%)		2,056 (8.28%)
&2B96		LDA #&00		4,112 (0.79%)	4,112 (0.79%)		2,056 (8.28%)
&2B98		RTS		12,336 (2.37%)	12,336 (2.37%)		2,056 (8.28%)

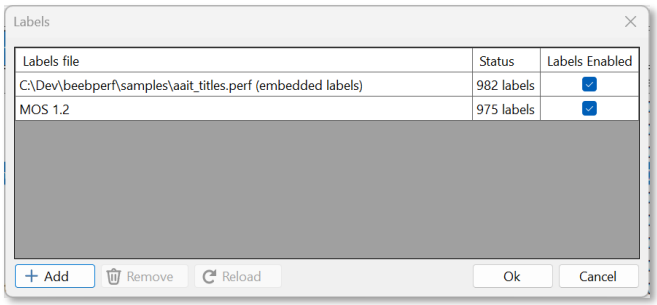
The code panel shows the executed instructions of the currently selected routine. We'll give more details on this later, but for now notice how tail calls and fall throughs are shown.

Labels...

Labels play a vital role in helping one understand the code. Without them, one just sees plain addresses.

In situations where **.perf** files don't have embedded labels (the sample **.perf** files have embedded labels), labels can be imported (side-loaded) using the **Labels** dialog, which is shown by clicking on the **Labels** toolbar button.

Note that labels are also automatically loaded and shown for MOS 1.2 and 2.0 code.



If you are working on multiple projects, you can have multiple labels files loaded and choose which ones you would like to be enabled or active. The list of imported label files persists across application sessions.

Labels can be imported from label files created using **BeebAsm.exe -labels <file>** command-line option, or from assembler listing files created by **BeebAsm**, **Baron**, and **Acme**.

See **docs/BeebPerf.doc** to learn how to embed labels in recorded **.perf** files.

Note that if an address has multiple labels associated with it, the expected label may not be shown.

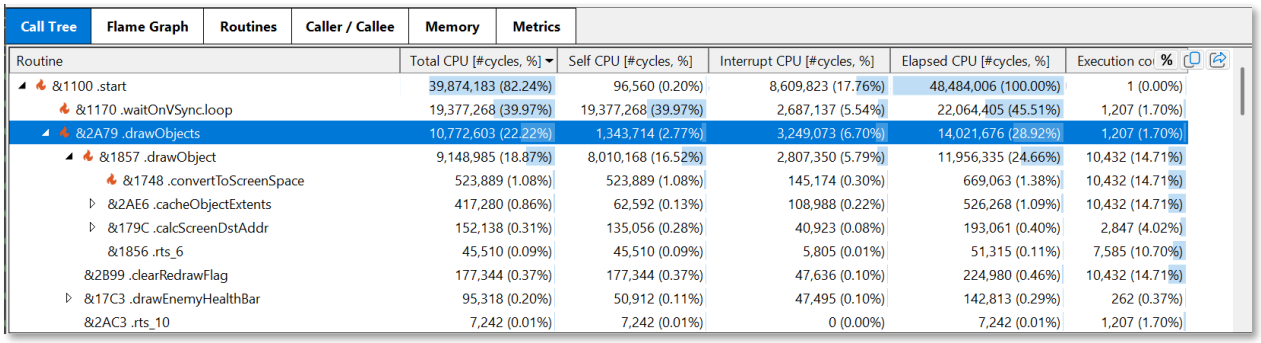
Investigations...

Where does the code spend most of its time?

Before diving into the tabs... the first thing to do is set the analysis time range so we're analyzing the right code. For example, if you want to analyze where the code is spending its time during game play, make sure the time range covers game play, and not something else.

There are several different ways to look at where time is being spent, each useful in their own way.

Click on the **Hot Paths** toolbar button to show the hot call paths.



Routine	Total CPU [#cycles, %]	Self CPU [#cycles, %]	Interrupt CPU [#cycles, %]	Elapsed CPU [#cycles, %]	Execution co %
&1100 .start	39,874,183 (82.24%)	96,560 (0.20%)	8,609,823 (17.76%)	48,484,006 (100.00%)	1 (0.00%)
&1170 .waitOnVSync.loop	19,377,268 (39.97%)	19,377,268 (39.97%)	2,687,137 (5.54%)	22,064,405 (45.51%)	1,207 (1.70%)
&2A79 .drawObjects	10,772,603 (22.22%)	1,343,714 (2.77%)	3,249,073 (6.70%)	14,021,676 (28.92%)	1,207 (1.70%)
&1857 .drawObject	9,148,985 (18.87%)	8,010,168 (16.52%)	2,807,350 (5.79%)	11,956,335 (24.66%)	10,432 (14.71%)
&1748 .convertToScreenSpace	523,889 (1.08%)	523,889 (1.08%)	145,174 (0.30%)	669,063 (1.38%)	10,432 (14.71%)
&2AE6 .cacheObjectExtents	417,280 (0.86%)	62,592 (0.13%)	108,988 (0.22%)	526,268 (1.09%)	10,432 (14.71%)
&179C .calcScreenDstAddr	152,138 (0.31%)	135,056 (0.28%)	40,923 (0.08%)	193,061 (0.40%)	2,847 (4.02%)
&1856 .rts_6	45,510 (0.09%)	45,510 (0.09%)	5,805 (0.01%)	51,315 (0.11%)	7,585 (10.70%)
&2B99 .clearRedrawFlag	177,344 (0.37%)	177,344 (0.37%)	47,636 (0.10%)	224,980 (0.46%)	10,432 (14.71%)
&17C3 .drawEnemyHealthBar	95,318 (0.20%)	50,912 (0.11%)	47,495 (0.10%)	142,813 (0.29%)	262 (0.37%)
&2AC3 .rts_10	7,242 (0.01%)	7,242 (0.01%)	0 (0.00%)	7,242 (0.01%)	1,207 (1.70%)

This selects the **Call Tree tab** which shows a hierarchical, top-down view of executed code paths, showing exactly which routines called which, how long they ran, and how often they were called etc.

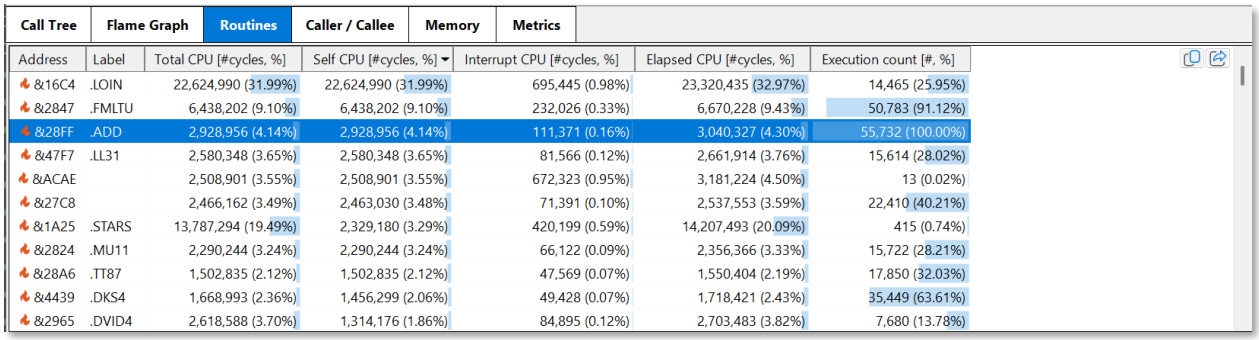
The call tree shows all the routine's call paths, sorted by **Total CPU** cycle counts. The top-level nodes represent the program and interrupt entry points. Each node in the tree shows metrics for the context in which it was invoked. Flame icons are displayed against routines in the hot call paths.

Make sure you scroll down as some hot paths may not be visible.

Clicking on a routine makes it the selected routine.

There are two ways to think about optimizing routines in the hot paths. Firstly, do routines in the hot path need to be called as often as they are? And secondly, are there ways to optimize the routines themselves?

The next way to look at where the time is being spent is to click on the **Hot Routines** toolbar button to show the hot routines.



Address	Label	Total CPU [#cycles, %]	Self CPU [#cycles, %]	Interrupt CPU [#cycles, %]	Elapsed CPU [#cycles, %]	Execution count [#, %]
&16C4	.LOIN	22,624,990 (31.99%)	22,624,990 (31.99%)	695,445 (0.98%)	23,320,435 (32.97%)	14,465 (25.95%)
&2847	.FMLTU	6,438,202 (9.10%)	6,438,202 (9.10%)	232,026 (0.33%)	6,670,228 (9.43%)	50,783 (91.12%)
&28FF	.ADD	2,928,956 (4.14%)	2,928,956 (4.14%)	111,371 (0.16%)	3,040,327 (4.30%)	55,732 (100.00%)
&47F7	.LL31	2,580,348 (3.65%)	2,580,348 (3.65%)	81,566 (0.12%)	2,661,914 (3.76%)	15,614 (28.02%)
&ACAE		2,508,901 (3.55%)	2,508,901 (3.55%)	672,323 (0.95%)	3,181,224 (4.50%)	13 (0.02%)
&27C8		2,466,162 (3.49%)	2,463,030 (3.48%)	71,391 (0.10%)	2,537,553 (3.59%)	22,410 (40.21%)
&1A25	.STARS	13,787,294 (19.49%)	2,329,180 (3.29%)	420,199 (0.59%)	14,207,493 (20.09%)	415 (0.74%)
&2824	.MU11	2,290,244 (3.24%)	2,290,244 (3.24%)	66,122 (0.09%)	2,356,366 (3.33%)	15,722 (28.21%)
&28A6	.TT87	1,502,835 (2.12%)	1,502,835 (2.12%)	47,569 (0.07%)	1,550,404 (2.19%)	17,850 (32.03%)
&4439	.DKS4	1,668,993 (2.36%)	1,456,299 (2.06%)	49,428 (0.07%)	1,718,421 (2.43%)	35,449 (63.61%)
&2965	.DVID4	2,618,588 (3.70%)	1,314,176 (1.86%)	84,895 (0.12%)	2,703,483 (3.82%)	7,680 (13.78%)

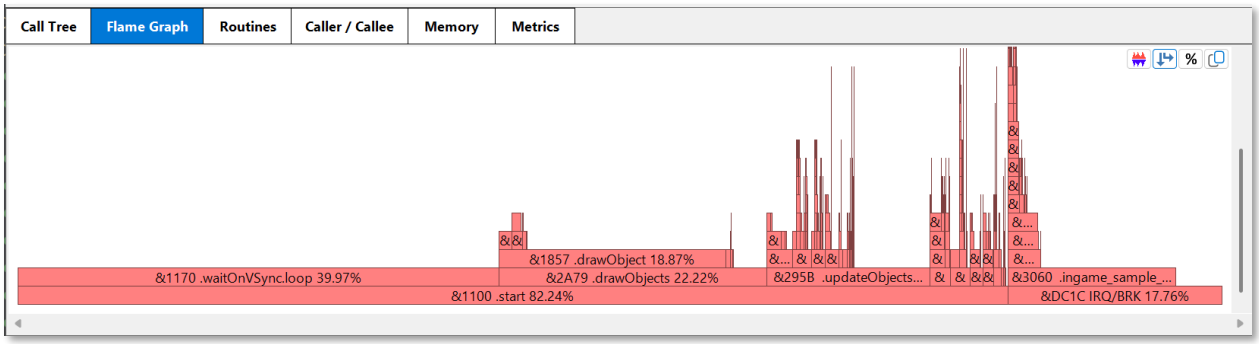
This selects the **Routines tab** which shows the routines in the program, sorted by their **Self-CPU** cycle counts. A flame icon is displayed against the 'hot' ones, those with the highest Self-CPU counts.

Clicking on a routine makes it the selected routine.

Unlike the Call Tree, where metrics are displayed within the context of the caller, the metrics shown in the Routines tab are aggregated across all callers.

The focus of this view is about the amount of time code executed just within the routines themselves. Focusing efforts on optimizing the code in the Hot Routines is simpler, and will lead to good performance improvements, but might miss some opportunities which are specific to the context in which the routine is invoked.

The last way to look at where the time is being spent is to click on the **Flame Graph tab**.



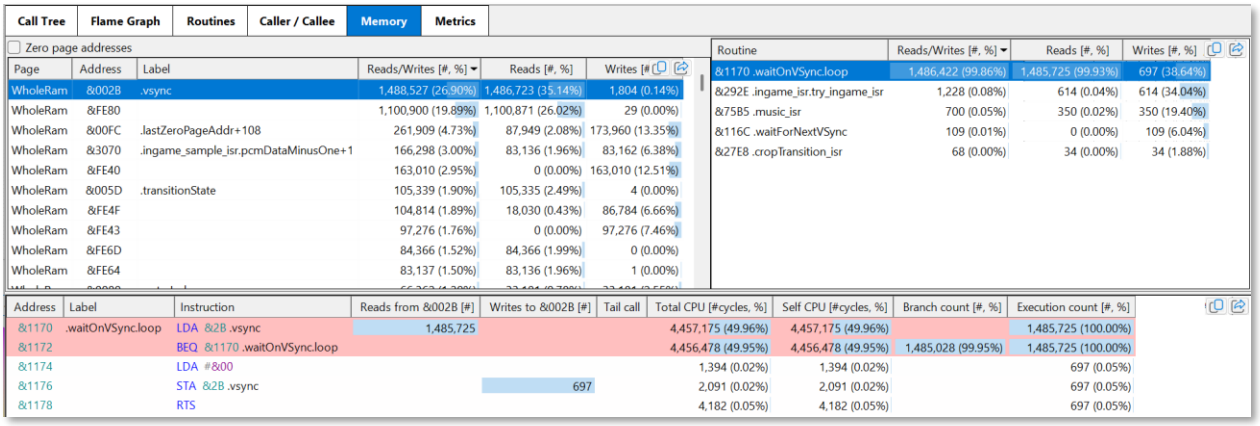
The **Flame Graph tab** shows a percentage breakdown of the call graph, where siblings are sorted by their Total CPU cycles.

The view provides a general visualization of where time is being spent, and where the bottlenecks are. It is also useful in highlighting oddities.

Clicking on a routine makes it the selected routine. Clicking in white space clears the selection.

Which memory addresses are accessed the most?

Click on the **Memory Tab** to show which memory addresses are accessed the most.



The memory tab lists the highest to lowest accessed memory addresses in the application. The memory can optionally be filtered to just zero-page addresses.

Ideally, the most frequently accessed memory addresses should be in zero-page.

Inspecting the highest accessed memory addresses is a useful exercise as it may reveal that certain addresses are accessed far more frequently than expected, indicating a potential bug. Inspection may also reveal that one or more of the accesses are not in zero-page, providing an optimization opportunity.

An instruction that uses zero-page addressing uses less CPU cycles and code space than the same instruction does using non zero-page addressing modes.

Look for opportunities where frequent non zero-page memory accesses can be changed to zero-page accesses.

Selecting an address populates the panel on the right with all the routines which read or write to the selected address, with their respective read and write counts. Selecting a routine from the list selects the routine in the code panel, in which the read and write counts are also shown against individual instructions.

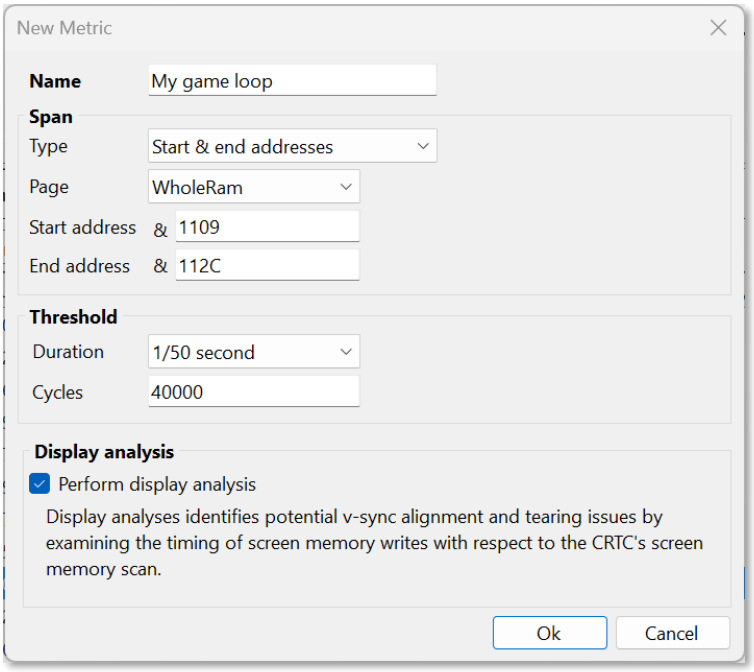
How is the game-loop behaving?

BeebPerf lacks the smarts to identify a game's game-loop, but it does provide a feature, '**Metrics**', accessible through the **Metrics tab**, that can be used for analyzing the game-loop.

The **Metrics tab** allows the creation of user definable metrics which allow one to gain insight into how specific routines, or **spans of code**, behave across iterations.

Metrics are created by clicking on the **Add...** button, naming the metric and setting its properties.

The following image shows the metrics settings for *An Adventure in Time's* game loop...



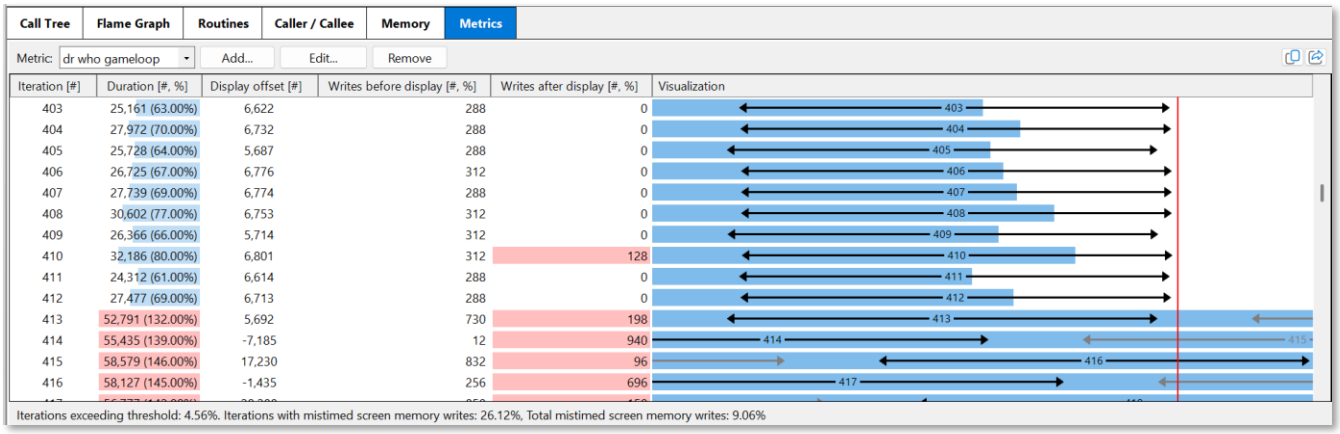
The **start address** is set to just after the game's WaitForVSync routine returns (just after v-sync is signaled) and the **end address** is set to the end of the game loop, when all processing has completed.

The **threshold** is set to 1/50th of a second and the **Perform display analysis** checkbox is also checked. Checking this box is useful for identifying potential v-sync alignment and display tearing issues.

V-Sync alignment issues occur when preceding iterations take longer than 1/50th of a second, causing the next game-loop iteration to start sometime after v-sync occurred.

Display tearing occurs when screen memory writes occur after the CRTC's screen memory scan reads the address. These writes will appear on the next frame, which may result in display tearing.

The following image shows some of the generated iterations.



You can see that many of the game-loop iterations look healthy, with processing times taking less than 1/50th of a second, and the corresponding display frames neatly right aligned. However, you can also see that some iterations...

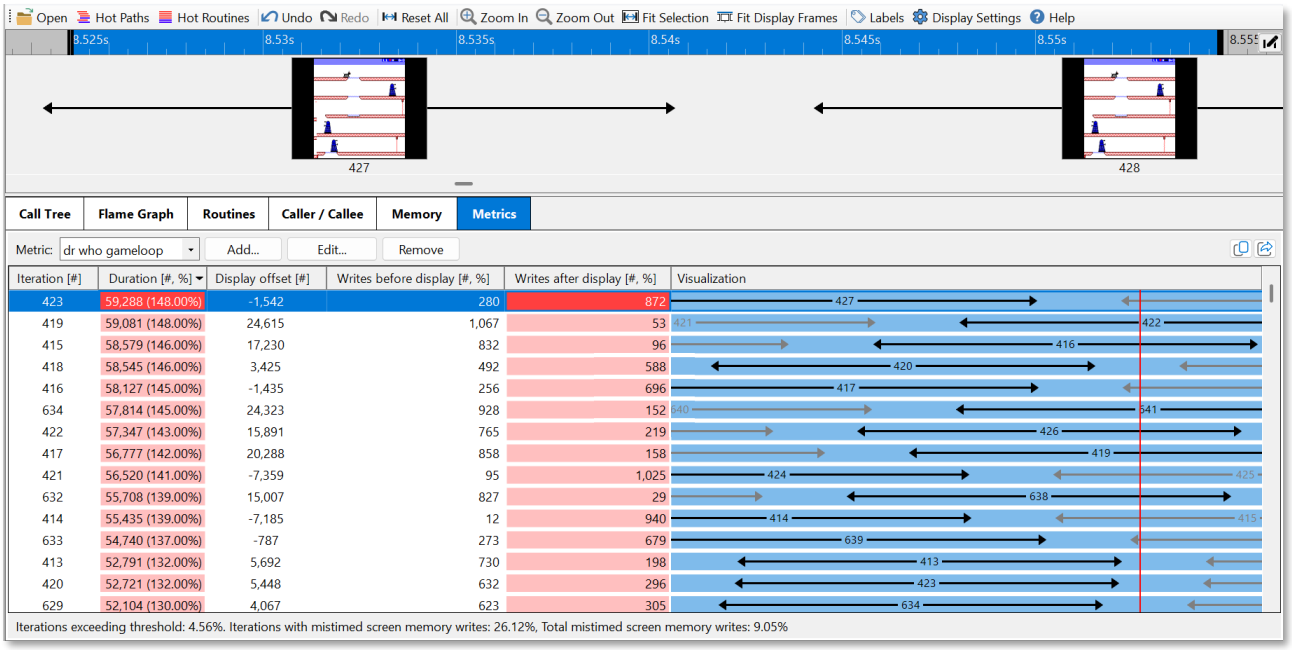
- Took longer than 1/50th of a second to process, and these are highlighted in the second column.

- Contained mistimed screen memory writes, and these are also highlighted in the fifth column.
- Are misaligned with the display frames, represented by the arrows which show the duration of the CRTC's screen memory scan of the visible area.

You can also see at the bottom of the image, that the summary line states that 4.56% of iterations exceeded the threshold (set to 1/50th of a second), and that 26.12% of iterations contained mistimed screen memory writes, and that overall, 9.05% of all screen memory writes were mistimed.

The net here is that further optimizations may be needed.

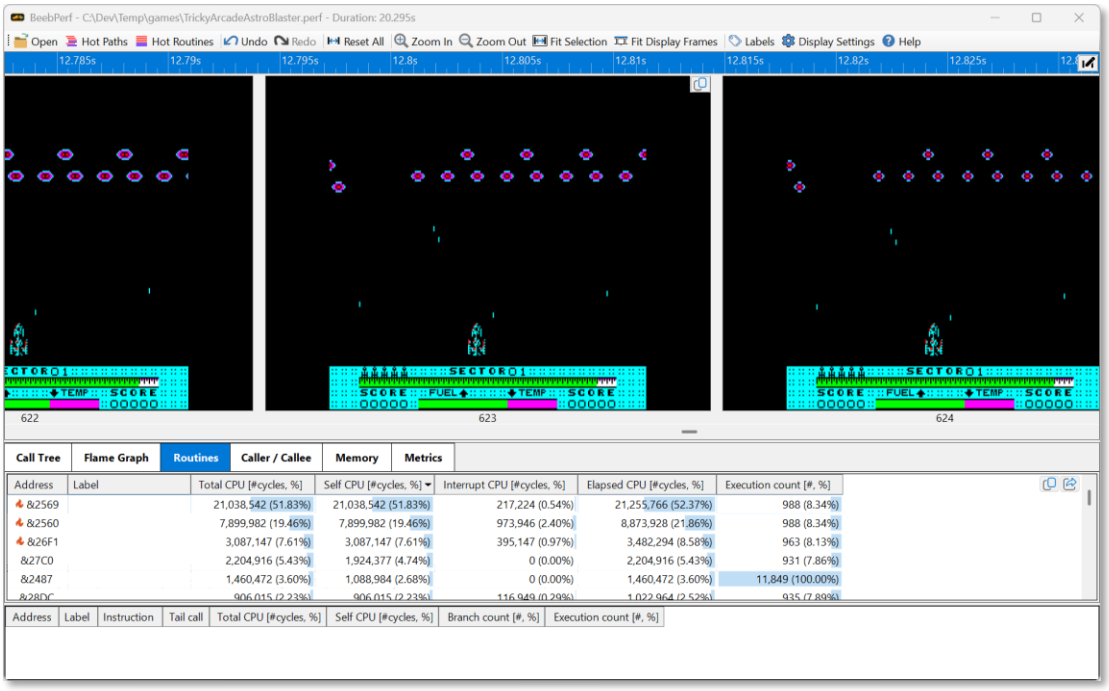
A good approach to take in this situation is to sort the iterations by their duration and select the longest one. This sets the analysis time range to just this iteration, as shown below.



You can now use the **Call Graph tab**, **Flame Graph tab**, and **Routines tab** to understand exactly that the code is doing during the longest game-loop iteration, focusing your optimization effort there.

What do the display frames look like?

The simplest way to look at the display frames is to resize the timeline panel so it's larger and press the **Fit Display Frames** toolbar button. This tiles the display frames across the window. One can then scroll through and examine them.

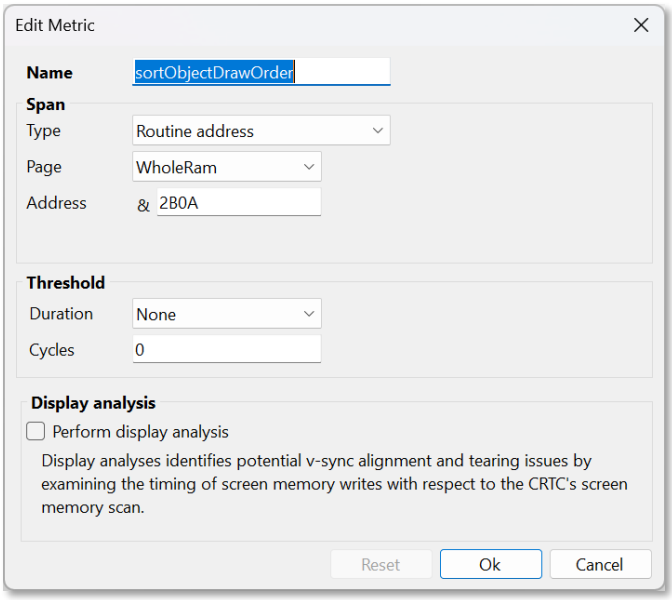


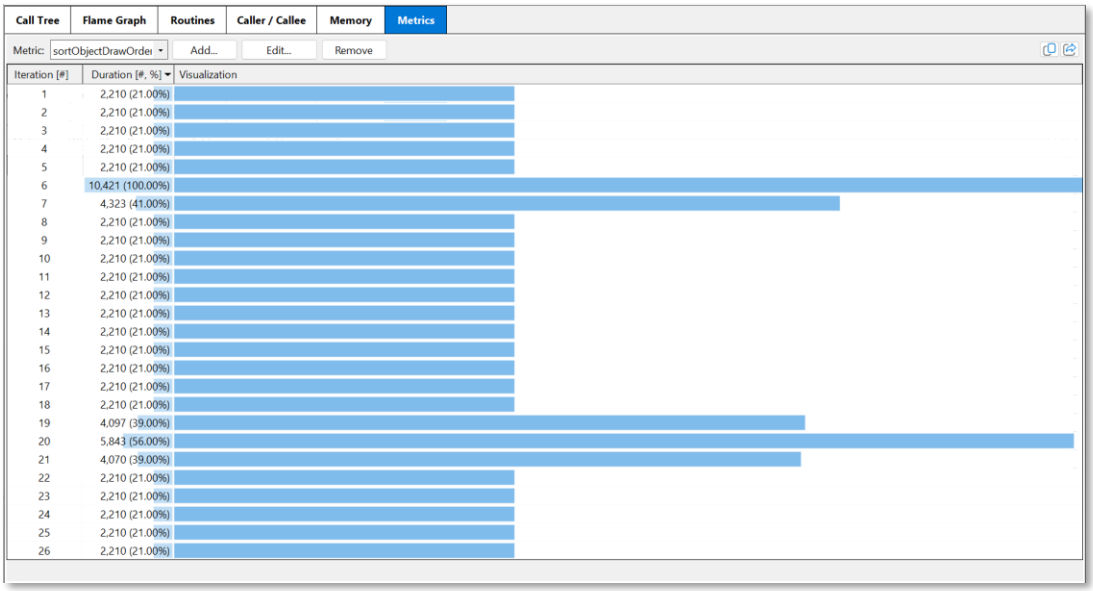
If you find a display frame that doesn't look correct you can examine the code that generated it using the same techniques we used to review game-loop iterations. You just need to find the game-loop iteration that generated the frame; and select it.

How does a routine's duration fluctuate over time?

The **Metrics tab** can also be used to analyze how a routine's durations fluctuate over time by creating a metric of **Type → Routine address, Threshold → none**, and **Perform Display Analysis →** unchecked.

For example, the following metric shows the iterations for *An Adventure in Time's* **sortObjectDrawOrder** routine which is called every thirty-two frames to sort the game objects by their screen Y position.





More details ...

The code panel

The code panel shows the executed instructions for the current selected routine, for the context in which it was selected, and for the selected analysis time range.

Address	Label	Instruction	Tail call	Total CPU [#cycles, %]	Self CPU [#cycles, %]	Branch count [#, %]	Execution count [#, %]
&295B	.updateObjects	LDX #&FF		2,414 (0.04%)	2,414 (0.04%)		1,207 (2.08%)
&295D		STX &7E .type		3,621 (0.05%)	3,621 (0.05%)		1,207 (2.08%)
&295F		INX		2,414 (0.04%)	2,414 (0.04%)		1,207 (2.08%)
&2960	.updateObjects.loop	STX &80 .outerIndex		173,838 (2.64%)	173,838 (2.64%)		57,946 (100.00%)
&2962		LDA &0400 .objectType,X		231,784 (3.52%)	231,784 (3.52%)		57,946 (100.00%)
&2965		AND #&03		115,892 (1.76%)	115,892 (1.76%)		57,946 (100.00%)
&2967		CMP &7E .type		173,838 (2.64%)	173,838 (2.64%)		57,946 (100.00%)
&2969		BEQ &2970 .updateObjects.skip		169,009 (2.57%)	169,009 (2.57%)	53,117 (91.67%)	57,946 (100.00%)
&296B		STA &7E .type		14,487 (0.22%)	14,487 (0.22%)		4,829 (8.33%)
&296D		JSR &2AEB .cacheObjectTypeExtents		193,160 (2.93%)	28,974 (0.44%)		4,829 (8.33%)
&2970	.updateObjects.skip	LDY &0580 .objectFrameCallback,X		231,784 (3.52%)	231,784 (3.52%)		57,946 (100.00%)
&2973		LDA &298A .updateObjects.frameCallbackTbl,Y		231,784 (3.52%)	231,784 (3.52%)		57,946 (100.00%)
&2976		STA &2980 .updateObjects.jsr_inst+1		231,784 (3.52%)	231,784 (3.52%)		57,946 (100.00%)
&2979		LDA &298B .updateObjects.frameCallbackTbl+1,Y		231,780 (3.52%)	231,780 (3.52%)		57,945 (100.00%)
&297C		STA &2981 .updateObjects.jsr_inst+2		231,780 (3.52%)	231,780 (3.52%)		57,945 (100.00%)
&297F	.updateObjects.jsr_inst	JSR &208D .enemyFrameCallback_MoveRight		777,905 (11.81%)	29,682 (0.45%)		4,947 (8.54%)
		JSR &22C8 .elevatorFrameCallback		1,214,554 (18.44%)	166,020 (2.52%)		27,670 (47.75%)
		JSR &2546 .teleportFrameCallback		107,549 (1.63%)	14,490 (0.22%)		2,415 (4.17%)
		JSR &255B .doorFrameCallback		587,831 (8.93%)	57,936 (0.88%)		9,656 (16.66%)
		JSR &2B79 .frameCallback_IntermittentRedraw		15,474 (0.23%)	3,540 (0.05%)		590 (1.02%)
		JSR &1F99 .enemyFrameCallback_LookRight		47,124 (0.72%)	7,392 (0.11%)		1,232 (2.13%)
		JSR &2039 .enemyFrameCallback_MoveLeft		586,173 (8.90%)	21,894 (0.33%)		3,649 (6.30%)
		JSR &1F6A .enemyFrameCallback_LookLeft		200,398 (3.04%)	30,606 (0.46%)		5,101 (8.80%)
		JSR &1F16 .enemyFrameCallback_TurnRightToLeft		13,632 (0.21%)	1,680 (0.03%)		280 (0.48%)
		JSR &1EC6 .enemyFrameCallback_TurnLeftToRight		13,384 (0.20%)	1,680 (0.03%)		280 (0.48%)
		JSR &1C1F .k9FrameCallback_MoveRight		56,610 (0.86%)	2,616 (0.04%)		436 (0.75%)
		JSR &2989 .updateObjects.return		16,920 (0.26%)	8,460 (0.13%)		1,410 (2.43%)
		JSR &22FD .elevatorFrameCallback_Descend		38,169 (0.58%)	576 (0.01%)		96 (0.17%)
		JSR &2417 .elevatorFrameCallback_Stop		2,449 (0.04%)	12 (0.00%)		2 (0.00%)
		JSR &1C58 .k9FrameCallback_TurnRightToLeft		2,963 (0.04%)	72 (0.00%)		12 (0.02%)
		JSR &1BF9 .k9FrameCallback_MoveLeft		21,422 (0.33%)	942 (0.01%)		157 (0.27%)
		JSR &1C40 .k9FrameCallback_TurnLeftToRight		686 (0.01%)	72 (0.00%)		12 (0.02%)
&2982		LDX &80 .outerIndex		173,838 (2.64%)	173,838 (2.64%)		57,946 (100.00%)
&2984		INX		115,892 (1.76%)	115,892 (1.76%)		57,946 (100.00%)
&2985		CPX &27 .objectCount		173,838 (2.64%)	173,838 (2.64%)		57,946 (100.00%)
&2987		BNE &2960 .updateObjects.loop		172,631 (2.62%)	172,631 (2.62%)	56,739 (97.92%)	57,946 (100.00%)
		fall-through					
&2989	.updateObjects.return	RTS		7,242 (0.11%)	7,242 (0.11%)		1,207 (2.08%)

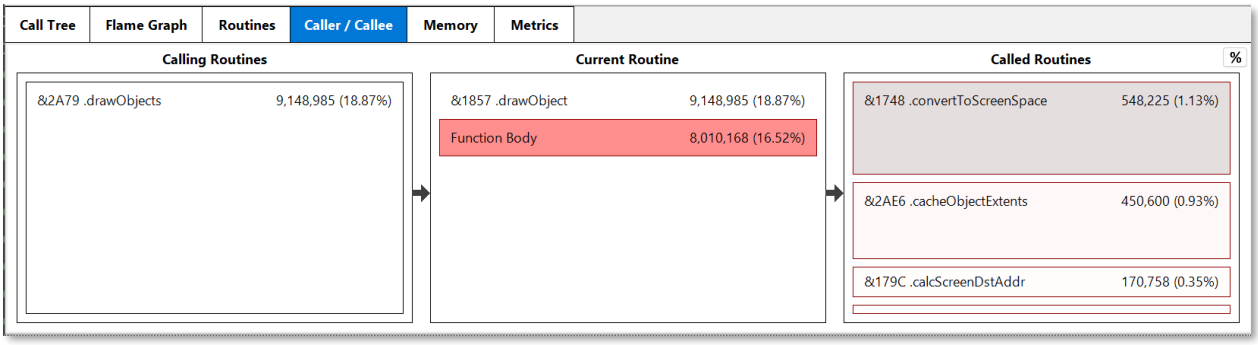
If alternative opcode sequences exist for an address, a vertical bar denotes that the code was modified, as shown above.

The background color shows which instructions take the most **Total CPU** cycles, compared to the other instructions. The table below describes the meaning of the different columns.

Column	Description
Tail call	Whether a JMP or branch instruction invokes a tail call. A tail call is a JMP or branch instruction that jumps to another routine which returns with a RTS or RTI. For a JMP, it's equivalent to a **JSR *routine* : RTS**, but takes up less code bytes. For a branch it's an optimized way to perform a conditional **JSR *routine* : RTS**
Total CPU	Number of cycles spent executing the instruction, including cycles spent in invoked routines (directly or indirectly) from JSR's and tail-calls. Excludes the cycles spent executing interrupt code during execution. The percentage shown is with respect to the total number of cycles executed by the routine, including cycles spent in invoked routines (directly or indirectly) from JSR's and tail-calls.
Self CPU	Number of cycles spent executing the instruction, including cycles spent in invoked routines (directly or indirectly) from JSR's and tail-calls. Excludes the cycles spent executing interrupt code during execution. The percentage shown is with respect to the total number of cycles executed by the routine, including cycles spent in invoked routines (directly or indirectly) from JSR's and tail-calls.
Branch count	The number of times the branch was taken, and the percentage of times the branch was taken.
Execution count	The number of times the instruction was executed.

Note that the visual appearance of the code can be changed by clicking the **Display Settings** toolbar button.

The Caller \ Callee tab

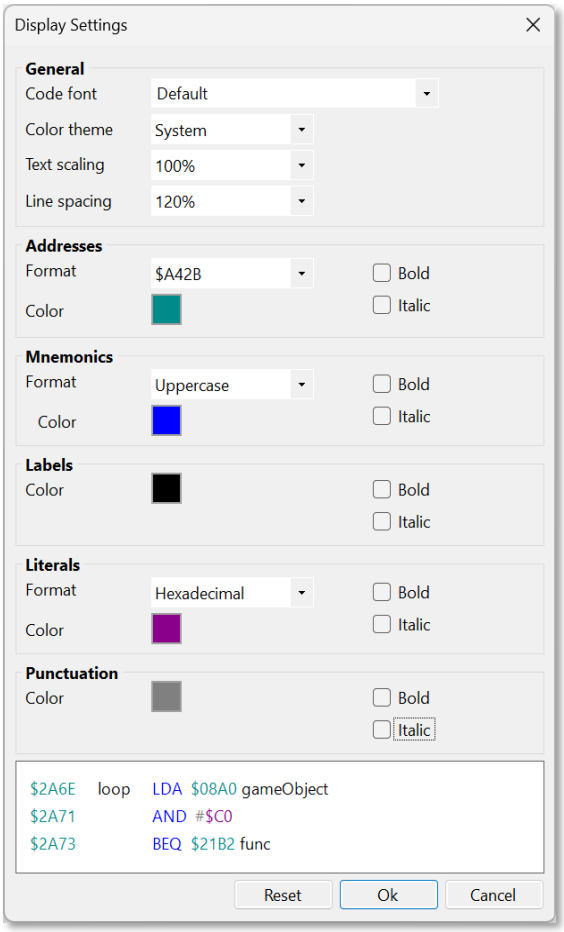


The **Caller / Callee tab** is useful for visualizing the callers and callees of the selected routine.

It show a routine's calling and called routines, showing their total CPU cycles and percentages with respect to the total number of cycles in the analysis time range.

Selecting a caller or callee makes it the currently selected routine.

The Display Settings dialog



The Display Settings dialog, shown by clicking the **Display Settings** toolbar button, allows you to adjust how code looks, how addresses are formatted, and allows you to set the application's theme (e.g. dark mode).

Acknowledgements

Portions of the display reconstruction code were based on **BeebEm's video** class. Many thanks to the **BeebEm** contributors for creating and maintaining this wonderful piece of software.

The **.perf** files are compressed and uncompressed using the **ZLib** library - Many thanks to Jean-loup Gailly, Mark Adler, and the other contributors, for creating and maintaining such an excellent and useful library.

The MOS 1.2 and 2.0 labels were taken from Toby Nelson's **MOS Reassembly for the BBC Micro**. Thank you Toby for creating and maintaining this excellent resource.

And many thanks to Alec Barker and Mike Hayton for their feedback and help on the initial version. Much appreciated.